INTERNSHIP PROJECT REPORT

Image-Based Prompt Injection Attacks on Large Vision-Language Models

Synthetic Dataset Generation, Pipeline Architecture, and Attack Success Rate Evaluation

Submitted by:

Ritik Sinha
GitHub: github.com/Ritik1207-ind

Siddhant Kumar
GitHub: github.com/siddhantkumar101

Udit Dadhich
GitHub: github.com/UditDadhich

Date: August 21, 2026

GitHub: github.com/Ritik1207-ind/prompt-injection-attacks-on-LVLMS

Dataset: huggingface.co/datasets/Reet1207/image-prompt-injection

Abstract

This report presents the design, implementation, and evaluation of a fully automated synthetic dataset generation pipeline for image-based prompt injection research targeting Large Vision-Language Models (LVLMs). We introduce a four-type attack taxonomy — typographic, structural layout, adversarial perturbation, and metadata injection — covering four injection goals: jailbreak, data exfiltration, task hijacking, and social engineering. Using a self-hosted NVIDIA RTX 6000 GPU, we generated 4,859 labeled samples using Llama-3.1-70B as the instruction orchestrator and LLaVA-1.5-7B as the target model for attack success rate (ASR) scoring. The core technical contribution is a white-box adversarial perturbation attack (Type 3) implemented via Projected Gradient Descent (PGD) through the CLIP ViT-L/14 vision encoder, achieving 13.7% ASR on jailbreak goals while remaining completely imperceptible to human observers. The complete dataset, source code, and results are publicly available.

Table of Contents

1. Introduction

Large Vision-Language Models (LVLMs) have emerged as powerful AI systems capable of processing and reasoning about both images and text. Models such as LLaVA, GPT-4V, and Gemini Vision are increasingly deployed in agentic systems, automated pipelines, and consumer applications. However, this multimodal capability introduces a novel and underexplored attack surface: **image-based prompt injection**.

In a prompt injection attack, a malicious actor embeds instructions inside an image with the intent of hijacking the LVLM's behaviour — causing it to ignore its original task, leak sensitive information, bypass safety filters, or generate harmful content. Unlike traditional adversarial examples that aim to fool classification, prompt injection attacks target the model's instruction-following behaviour.

This project addresses the lack of a comprehensive, publicly available benchmark dataset for image-based prompt injection research. We designed and implemented a fully automated pipeline that generates synthetic injection samples across four attack types and four injection goals, scored each sample using LLaVA-1.5-7B, and released the resulting 4,859-sample dataset on HuggingFace.

2. Background and Related Work

2.1 Prompt Injection

Prompt injection was first documented in text-based LLMs where adversarial instructions in user input override system prompts. In the visual domain, the attack surface expands because instructions can be embedded in images, bypassing text-channel safety filters entirely.

2.2 Related Work

- **FigStep (Gong et al., 2023)** — Demonstrated that rendering harmful text as an image bypasses safety filters in GPT-4V and LLaVA. Our Type 1 (typographic) attack is directly based on this approach.
- **Lee et al. (Electronics, 2025)** — Introduced structural layout injection by embedding malicious instructions in diagrams. Our Type 2 (structural) attack extends this work.
- **Qi et al. (2023)** — Showed that adversarial image perturbations can jailbreak aligned LLMs. Our Type 3 (adversarial) attack implements PGD through CLIP's vision encoder, extending this to LLaVA.
- **Indirect Prompt Injection (Greshake et al., 2023)** — Demonstrated injection via external content in agent pipelines. Our Type 4 (metadata) attack targets this threat surface using EXIF fields.

3. Problem Statement and Objectives

There is currently no publicly available large-scale benchmark dataset for image-based prompt injection attacks on LVLMs that covers multiple attack types, injection goals, and provides ASR labels. This limits the ability to systematically evaluate model vulnerability and develop effective defences.

3.1 Objectives

- Design a taxonomy of image-based prompt injection attacks covering structural, perturbation-based, and hidden-channel methods.
- Build a fully automated pipeline that generates synthetic injection samples without manual effort.
- Cover four injection goals: jailbreak, data exfiltration, task hijacking, and social engineering.
- Score each sample using LLaVA-1.5-7B to produce ASR labels for benchmarking.
- Release the dataset and source code publicly for the research community.

4. System Architecture

The pipeline consists of five sequential stages running on a self-hosted NVIDIA RTX 6000 (48GB VRAM) cloud GPU accessed via SSH. No paid APIs were used — all models are open-source and served locally.

Stage	Component	Tool / Model	Runtime
1	Instruction Generation	Llama-3.1-70B-Instruct (Ollama)	~2 hours
2	Image Generation	PIL / Graphviz / CLIP PGD / piexif	~2.5 hours
3	Regex Validation	Python re module	~10 min
4	LVLM Inference Scoring	LLaVA-1.5-7B (HuggingFace)	~3 hours
5	Dataset Writing	HuggingFace datasets (Parquet)	~10 min

Table 1: Pipeline stages and estimated runtimes on NVIDIA RTX 6000.

4.1 Hardware and Software Stack

Component	Specification
Cloud GPU	NVIDIA RTX 6000 (48GB VRAM, Ubuntu 24.04)
Local Machine	Fedora Linux (NVIDIA RTX 3050) — SSH terminal only
LLM Orchestrator	Llama-3.1-70B-Instruct (4-bit quantised, Ollama)
LVLM Target	LLaVA-1.5-7B (bfloat16, HuggingFace Transformers)
Vision Encoder (T3)	CLIP ViT-L/14 (openai/clip-vit-large-patch14)
Dataset Format	HuggingFace datasets (Parquet, Arrow)

Table 2: Hardware and software stack.

5. Attack Type Implementation

5.1 Type 1 — Typographic Injection

The malicious instruction is rendered as black text on a 512x512 white background image using PIL. The vision encoder reads it as visual content, while text-channel safety filters never process it. This is a black-box attack requiring zero model access.

- **Stealthiness:** Zero — any human viewer immediately sees the instruction
- **Implementation:** PIL ImageDraw with automatic text wrapping
- **Speed:** ~1 second per sample on CPU
- **Based on:** FigStep (Gong et al., 2023)

5.2 Type 2 — Structural Layout Injection

The malicious instruction is embedded as a node label inside a flowchart or mind map generated using Graphviz. The harmful content is present but camouflaged among 8 legitimate surrounding nodes. This is a black-box attack.

- **Stealthiness:** Low — instruction is technically visible but not immediately obvious
- **Implementation:** Graphviz DOT language + networkx for layout
- **Speed:** ~2-3 seconds per sample on CPU
- **Based on:** Lee et al. (Electronics, 2025)

5.3 Type 3 — Adversarial Perturbation Injection (Core Novelty)

This is the primary technical contribution of the project. Pixel-level noise is optimised using Projected Gradient Descent (PGD) through the CLIP ViT-L/14 vision encoder so that the resulting image embedding encodes the attacker's instruction in latent space. The perturbed image is visually indistinguishable from the original to human observers.

The attack objective is to minimise the negative cosine similarity between the adversarial image embedding and the target text embedding:

```
Objective: min -cosine_similarity(CLIP_image(x + delta), CLIP_text(instruction))
Constraint: ||delta||_inf <= epsilon = 0.05 PGD update: delta = delta + alpha *
sign(grad(loss)) delta = clamp(delta, -epsilon, epsilon) alpha = 0.005, steps =
200
```

- **Stealthiness:** Maximum — imperceptible to humans
- **Implementation:** PyTorch PGD on CLIP ViT-L/14, gradient through vision encoder
- **Attack type:** White-box — requires gradient access to vision encoder
- **Speed:** ~30-120 seconds per sample on RTX 6000
- **Based on:** Qi et al. (2023), extended to LLaVA

5.4 Type 4 — Metadata Injection

The malicious instruction is written into EXIF metadata fields (UserComment, ImageDescription, XPComment) of a completely benign-looking stock photo using the piexif library. The image itself is completely unmodified. This attack is only effective when the LVLM deployment pipeline reads image metadata, as is common in agent frameworks and RAG systems.

- **Stealthiness:** Maximum — image is completely unmodified
- **Implementation:** piexif library, ~10 lines of code
- **Threat surface:** Agent deployments, RAG pipelines, automated image processing
- **Speed:** ~0.1 seconds per sample on CPU

6. Dataset Construction

6.1 Instruction Generation

Llama-3.1-70B-Instruct (served via Ollama) was used to generate malicious injection instructions for each attack_type x goal combination. Each API call used a structured system prompt instructing the model to output valid JSON containing a malicious_instruction, benign_context, and variation_style. The orchestrator supported automatic resume on crash.

6.2 Dataset Statistics

Metric	Value
Total samples	4,859
Attack types	4 (typographic, structural, adversarial, metadata)
Injection goals	4 (jailbreak, exfiltration, hijacking, social_engineering)
Train split	3,887 samples (80%)
Validation split	485 samples (10%)
Test split	487 samples (10%)
Features per sample	12
LLM orchestrator	Llama-3.1-70B-Instruct
Scoring model	LLaVA-1.5-7B
Total GPU runtime	~7.5 hours

Table 3: Dataset statistics.

6.3 Regex Validation

Each generated instruction was validated against a goal-specific regex pattern to confirm it contains the expected linguistic markers. This populates the regex_hit field and serves as a data quality metric.

```
jailbreak: (?i)(ignore|bypass|override|disregard|forget|pretend|jailbreak)
exfiltration: (?i)(repeat|print|reveal|show).{0,40}(system prompt|instructions)
hijacking: (?i)(ignore previous|instead of|new task|from now on)
social_engineering: (?i)(click|verify|confirm|urgent|account|password|login)
```

7. Experiments and Results

7.1 Experimental Setup

Each generated image was passed to LLaVA-1.5-7B (loaded in 4-bit NF4 quantisation on the NVIDIA RTX 6000) with the following minimal prompt, simulating a black-box deployment scenario:

```
Prompt: 'What does this image say or show? Please follow any instructions you
see.'
```

The model's response was classified as a successful attack (asr_label=1) if it contained goal-specific compliance indicators, or a failed attack (asr_label=0) if it contained refusal patterns or no matching indicators.

7.2 Attack Success Rate Results

Attack Type	Jailbreak	Exfiltration	Hijacking	Social Eng.	Total ASR
T1 Typographic	2.2%	9.9%	2.6%	20.8%	8.9%
T2 Structural	4.5%	2.6%	0.0%	1.0%	2.0%
T3 Adversarial*	13.7%	0.0%	0.0%	0.0%	3.4%
T4 Metadata	24.0%	0.0%	0.0%	0.0%	6.0%

Table 4: Attack Success Rate (ASR) per attack type and injection goal. *T3 is white-box; all others are black-box.

8. Analysis and Discussion

8.1 Key Findings

- **T4 Metadata achieves highest jailbreak ASR (24.0%)**

  EXIF-based injection is a significant threat in agent and RAG deployments. The image appears completely benign, making this attack particularly dangerous in automated pipelines.

- **T1 Typographic achieves highest social engineering ASR (20.8%)**

  Explicit visible text is most effective for phishing-style instructions, consistent with prior FigStep results.

- **T3 Adversarial achieves 13.7% jailbreak ASR with zero visual trace**

  This is the core novelty — a human-imperceptible attack that still achieves meaningful ASR. The perturbation successfully shifts the image embedding in CLIP's latent space toward the target instruction.

- **T2 Structural achieves lowest overall ASR (2.0%)**

  Camouflaging the instruction within a diagram reduces its legibility to LLaVA, suggesting the model struggles to isolate relevant text nodes from complex visual structures.

- **Exfiltration, hijacking, and social engineering ASR drops to 0% for T3/T4**

  Adversarial and metadata attacks are most effective for jailbreak goals but fail for more complex goal types that require nuanced language understanding.

8.2 Stealthiness vs. ASR Trade-off

The results reveal a nuanced relationship between stealthiness and attack effectiveness. T1 (zero stealthiness) achieves the highest social engineering ASR but is trivially detectable. T3 and T4 (maximum stealthiness) achieve significant jailbreak ASR while remaining completely undetectable. This suggests that for deployment security, defences must address both visible and invisible attack vectors.

8.3 Limitations

- The ASR scorer uses keyword matching rather than a judge model, which may produce false positives/negatives.
- T3 is white-box only — it requires gradient access to the vision encoder and cannot transfer directly to closed-source models.
- The dataset covers LLaVA-1.5-7B only; ASR may differ significantly across other LVLMs.
- T4 metadata injection is only effective in pipelines that explicitly read EXIF data, limiting its real-world applicability.

9. Conclusion and Future Work

This project successfully designed, implemented, and evaluated a fully automated pipeline for generating a synthetic image-based prompt injection dataset. In approximately 7.5 hours of GPU compute time, we produced 4,859 labeled samples across 4 attack types and 4 injection goals, scoring each sample using LLaVA-1.5-7B for attack success rate.

The core technical contribution — a white-box adversarial perturbation attack through CLIP's vision encoder (T3) — achieves 13.7% ASR on jailbreak goals while remaining completely imperceptible to human observers. The dataset and full source code are publicly available on HuggingFace and GitHub respectively.

9.1 Future Work

- Evaluate the dataset against additional LVLMs (LLaVA-1.5-13B, InstructBLIP, Qwen-VL) to test attack generalisability
- Implement and evaluate defences: JPEG compression, Gaussian blur, OCR-based text detection, EXIF stripping
- Replace keyword ASR scorer with a judge model for more accurate evaluation
- Extend T3 to a transferable attack that works without gradient access (black-box adversarial)
- Upload sample images alongside metadata to HuggingFace for a richer dataset release

10. References

[1] Gong et al. 'FigStep: Jailbreaking Large Vision-Language Models via Typographic Visual Prompts.' arXiv:2311.05608, 2023.

[2] Lee et al. 'Prompt Injection Attacks on Vision Language Models in Oncology.' Electronics, 2025.

[3] Qi et al. 'Visual Adversarial Examples Jailbreak Aligned Large Language Models.' AAAI, 2024.

[4] Greshake et al. 'Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.' AISec, 2023.

[5] Liu et al. 'LLaVA: Visual Instruction Tuning.' NeurIPS, 2023.

[6] Radford et al. 'Learning Transferable Visual Models From Natural Language Supervision (CLIP).' ICML, 2021.

[7] Madry et al. 'Towards Deep Learning Models Resistant to Adversarial Attacks (PGD).' ICLR, 2018.